Agile Flight Using Neuroevolution

A Comparison of CMA-ES and NEAT

by

Przemyslaw Lesiczka

AE4350 – Bio-inspired Intelligence and Learning for Aerospace Applications

Student number:	5752884
Course:	AE4350
Instructors:	Prof.dr. G.C.H.E. de Croon, Dr.ir. E. van Kampen
Project Duration:	August 2026
Faculty:	Faculty of Aerospace Engineering, TU Delft

The code associated with this report can be found at:
https://github.com/plesiczka04/AE4350.

TU Delft Delft University of Technology

1. Introduction

At their performance limits, quadrotors can reach speeds over $80\,\mathrm{km\,h^{-1}}$ and accelerations above $4\,g$. In this regime the perception, planning, and control systems are all pushed to their limits and minor errors can compound quickly [1]. Designing traditional controllers for these speeds is difficult as vehicle dynamics become highly non-linear and aerodynamic effects are hard to model. As a result, learning-based controllers for tasks like drone racing have become increasingly popular.

Recent work has shown the potential of these approaches. Swift achieved human-champion performance in drone racing using deep reinforcement learning (RL) trained in simulation, bridging the reality gap with a learned residual model for unmodelled dynamics [2]. Similarly, Learning Agile Flight in the Real World (LAFR) [3] instead learns agile control directly on hardware using a differentiable simulator. LAFR uses adaptive temporal scaling to adjust the timing of a reference trajectory according to a closed-loop sensitivity estimate. The controller is therefore able to learn the quadcopter's limits.

A similar trade-off also appears in classical control. Minimum-snap trajectory generation is time-parameterised. Here, reducing the time available to fly a path increases velocity, acceleration, and control effort until the vehicle becomes unstable or the reference becomes infeasible [4]. Every agile-flight controller therefore selects an operating point which trades tracking accuracy and stability against speed. Learning-based methods differ mainly in how this operating point is found. This leads to the main question of this report:

> Does the agility–safety trade-off of a learned agile-flight controller also emerge under gradient-free evolutionary optimisation, and do a fixed-topology approach (CMA-ES) and a topology-evolving approach (NEAT) discover it differently?

To study this, a fixed-topology multilayer perceptron (MLP) optimised with Covariance Matrix Adaptation Evolution Strategy (CMA-ES) is compared with a topology-evolving network trained with Neuroevolution of Augmenting Topologies (NEAT). Both use the same simulation and fitness function, while the controller topology is fixed for CMA-ES and evolved for NEAT. The report examines the resulting agility–accuracy frontier, sensitivity to important hyperparameters, and sensitivity to disturbances.

2. Background and Related Work

In the past, quadrotor flight has used tuned model-based controllers combined with differential-flatness trajectory generation and geometric tracking [4, 5]. More recent learning-based methods have improved performance at the limit of the flight envelope, where unmodelled aerodynamics become important [1, 2]. Many of these methods use gradients, either through policy-gradient reinforcement learning [6], or direct backpropagation through a differentiable dynamics model, as seen in LAFR [3]. Regardless of the optimisation method, the control problem is to map the vehicle state and reference trajectory to a set of feasible commands. This report keeps the task and low-level control structure fixed and changes are only made to the optimiser.

A quadrotor can be considered an underactuated rigid body with six degrees of freedom and four independent actuators. Thrust usually acts along a single body axis. Its rotational dynamics are much faster than its translational ones and are only marginally stable, so a small rotor asymmetry can grow into a tumble rather than settling [4, 5]. This property explains why many quadrotor controllers are cascaded, with a fast inner loop regulating attitude and body rate and a slower outer loop regulating position. The same structure is used here because direct evolution of four raw motor speeds is beyond the scope of this report.

Neuroevolution trains neural networks with evolutionary algorithms [7]. Unlike gradient descent, it requires only a scalar fitness value and does not require a differentiable objective. Depending on the representation, evolution can modify network weights alone or both weights and topology. Evolution strategies have been shown to be competitive alternatives to gradient-based reinforcement learning for continuous control, although they can require more samples [8]. Evolutionary robotics extends the idea to complete robots but creates a reality gap, where a controller evolved in simulation can exploit simulation inaccuracies but fail in hardware-in-the-loop testing [9, 10]. In this report, this gap is only examined through disturbance tests. Neuroevolution has also been applied to aerospace control problems. Koppejan and Whiteson [11] evolved neural controllers for wind-varying helicopter hovering,

and Mariani and Fiori [12] evolved a multilayer-perceptron controller for simulated quadcopter path following. These studies demonstrate that evolutionary methods can produce competent controllers for quadcopters and other rotorcraft.

All experiments use gym-pybullet-drones [13], with the PyBullet physics engine. Its default CF2X model represents the Bitcraze Crazyflie 2.X using its mass, inertia, thrust, and drag coefficients. A lightweight physics simulator is appropriate for this evolutionary search approach as a single run evaluates many candidates and the full study requires a large number of rollouts. A more expensive simulator would reduce the throughput required by the evolutionary algorithms.

3. Method

The quadcopter is modelled as a rigid body with mass m, inertia J, position $\mathbf{r}$, and orientation $R \in SO(3)$. Positions and velocities are expressed in a fixed world frame $\mathcal{F}_W$ (ENU, z up); attitude R relates $\mathcal{F}_W$ to a body frame $\mathcal{F}_B$ at the centre of mass with thrust along $+z_B$, and body rates $\boldsymbol{\omega}$ are given in $\mathcal{F}_B$. The Newton–Euler dynamics are [4, 5]:

$$\begin{aligned} m\ddot{\mathbf{r}} &= -mg\,\mathbf{e}_3 + fR\mathbf{e}_3, \quad \dot{R} = R\hat{\boldsymbol{\omega}}, \\ J\dot{\boldsymbol{\omega}} &= \boldsymbol{\tau} - \boldsymbol{\omega} \times J\boldsymbol{\omega}, \end{aligned} \tag{1}$$

where f is the collective thrust, $\boldsymbol{\tau}$ is the body torque, and $\hat{(\cdot)}$ is the skew-symmetric cross-product map. Rotor i produces thrust $f_i = k_f\omega_i^2$ and reaction torque $k_m\omega_i^2$, and a fixed mixer converts collective thrust and body torque into rotor speeds. As thrust acts only along the body z-axis, horizontal acceleration is coupled to attitude. Given the marginal stability mentioned above and the difficulty of directly evolving four motor commands, a cascaded controller is used.

The reference trajectory is a horizontal figure-8 at fixed altitude $z_0 = 1\,\mathrm{m}$ and half-width $R = 2.5\,\mathrm{m}$:

$$\mathbf{r}_{\mathrm{ref}}(t) = \left(R\sin\omega t,\ \frac{R}{2}\sin 2\omega t,\ z_0\right), \qquad \omega = \frac{2\pi}{T}. \tag{2}$$

The lap time T is the agility parameter for trajectory tracking for which a smaller T requires the same path to be flown faster. Each episode contains two complete laps. The 5 m-wide track was chosen to keep the tested trajectories close to the vehicle's actuation limits without making the reference clearly infeasible.

The evolved network maps a 12-dimensional observation to three commands $\mathbf{a} = [a_T, a_\phi, a_\theta] \in [-1, 1]^3$. These commands specify thrust around hover and desired roll and pitch angles. A fixed inner attitude/rate controller converts them to motor RPM:

$$\boldsymbol{\tau} = K_P^{\tau}(\boldsymbol{\eta}^* - \boldsymbol{\eta}) - K_D^{\tau}\boldsymbol{\omega}, \qquad \boldsymbol{\omega}_{\mathrm{cmd}} = s\,(T^* + M\boldsymbol{\tau}) + c, \tag{3}$$

where $\boldsymbol{\eta}^* = [a_\phi\eta_{\max}, a_\theta\eta_{\max}, 0]$, M is the CF2X mixer, T^* is the thrust command, and (s, c) are the PWM-to-RPM conversion constants. The inner loop maintains attitude stability so that evolution mainly learns the mapping from tracking state to thrust and attitude as summarised in Figure 1.

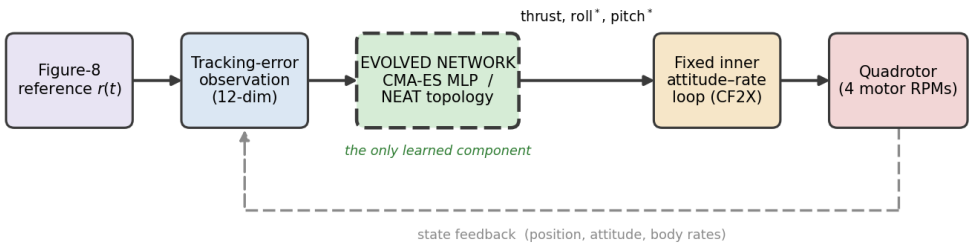


Figure 1: Cascaded neuroevolution controller with a fixed inner loop and evolved network

The observation is $\mathbf{o} = [\mathbf{e}_p, \mathbf{e}_v, \boldsymbol{\eta}, \boldsymbol{\omega}] \in \mathbb{R}^{12}$, where $\mathbf{e}_p = \mathbf{r}_{\text{ref}} - \mathbf{r}$ is the position error and $\mathbf{e}_v$ the velocity error. Fitness is based on mean position error, with optional control-effort and body-rate penalties:

$$\mathcal{F} = -\left(\frac{1}{N} \sum_k \|\mathbf{e}_p(k)\| + w_u \, \overline{u} + w_\omega \, \overline{\|\boldsymbol{\omega}\|} \right). \tag{4}$$

In the reported experiments $w_u = w_\omega = 0$, so tracking error is the optimisation objective and the control effort and body rates are recorded only as diagnostics. If a vehicle crashes because of ground contact, excessive tilt, or $\|\mathbf{e}_p\| > 2R$, the remaining steps receive the maximum error penalty. The zero-output network corresponds to stable hover and provides a common starting point.

3.1. CMA-ES

CMA-ES is a derivative-free optimiser for continuous, non-convex black-box problems [14, 15]. It maintains a Gaussian search distribution $\mathcal{N}(\mathbf{m}, \sigma^2 C)$ over the parameter vector. Each generation samples candidate vectors, ranks them by fitness, and updates the mean, covariance, and global step size:

$$\mathbf{m} \leftarrow \mathbf{m} + \sigma \sum_k w_k \mathbf{y}_k, \tag{5}$$

$$C \leftarrow (1 - c_1 - c_\mu) C + c_1 \mathbf{p}_c \mathbf{p}_c^\top + c_\mu \sum_k w_k \mathbf{y}_k \mathbf{y}_k^\top, \tag{6}$$

$$\sigma \leftarrow \sigma \exp\left[\frac{c_\sigma}{d_\sigma} \left(\frac{\|\mathbf{p}_\sigma\|}{\mathbb{E}\|\mathcal{N}(0, I)\|} - 1 \right) \right]. \tag{7}$$

Here $\mathbf{p}_c$ and $\mathbf{p}_\sigma$ are evolution paths. Covariance adaptation allows CMA-ES to learn dependencies between parameters and adjust the search direction. In this study, CMA-ES evolves the weights of a fixed 12–16–3 MLP with `tanh` activations using the `pycma` implementation. Only the network weights are optimised.

3.2. NEAT

NEAT evolves both network structure and weights [16]. Its main mechanisms are historical markings, speciation, and complexification. Historical innovation numbers allow networks with different structures to be aligned during crossover. Speciation protects new structures while their weights are tuned, using the compatibility distance

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \overline{W}, \tag{8}$$

where E and D are excess and disjoint genes, N is the genome size, and $\overline{W}$ is the mean weight difference between matching genes. NEAT starts with minimal networks and adds nodes and connections when they improve fitness. This allows the architecture to be discovered rather than fixed. The study uses neat-python with tanh nodes, 12 inputs, and 3 outputs. The resulting policy uses the same cascaded controller and fitness function as CMA-ES, so differences are due to the optimisation method and representation. The full procedure is summarised in Algorithm 1.

Algorithm 1 NEAT controller evolution

```
initialise minimal genomes (inputs → outputs)
for generation = 1 to G do
    F_i ← ROLLOUT(net(g_i)) for each genome g_i
    speciate by compatibility δ and apply fitness sharing
    reproduce within species using elitism and selection
    mutate weights, nodes, and connections
end for
return champion genome
```

4. Experimental Setup

The physics simulation runs at 240 Hz and the control loop at 48 Hz. Each episode contains two laps. The agility sweep uses lap times corresponding to peak commanded speeds from $1.5\,\mathrm{m\,s^{-1}}$ to $4.0\,\mathrm{m\,s^{-1}}$. The nominal operating point is $T = 7.4\,\mathrm{s}$, corresponding to approximately $3.0\,\mathrm{m\,s^{-1}}$. Unless stated otherwise, both CMA-ES and NEAT use a population of 24 for 200 generations. Each configuration is repeated over 5 random seeds, and results are reported as mean ± standard deviation. The main parameters and the values swept in the sensitivity study are listed in Table 1.

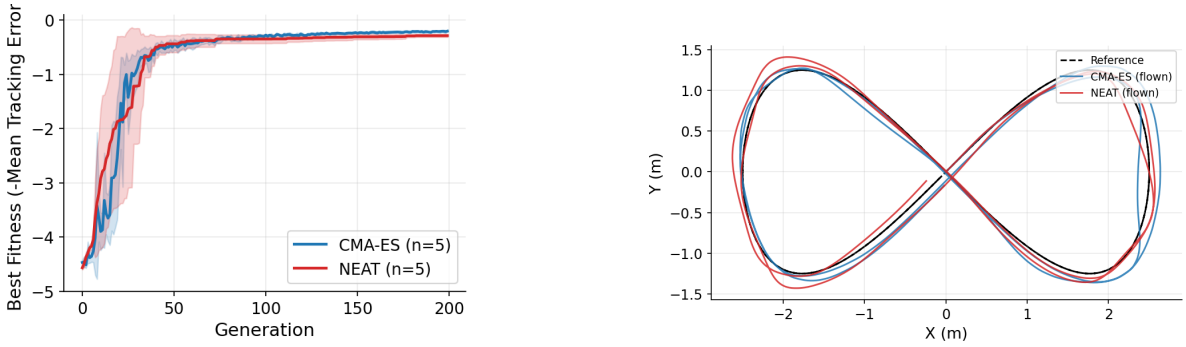
The sensitivity study varies CMA-ES initial step size, population size, and hidden-layer width, and NEAT population size. The complete study contains 123 evolutionary runs with 60 for the frontier and 63 for the sensitivity study. Runs are CPU-only and use seven parallel workers on a four-core/eight-thread CPU. The complete study required about 16 hours of clock time. Full configuration files are provided with the code.

Table 1: Main parameters varied in the sensitivity study with the defaults in bold

Environment & policy		*Evolutionary search*	
Physics / control rate	240 / 48 Hz	CMA-ES population λ	8, 16, **24**, 32, 48
Episode length	2 laps	CMA-ES initial step σ_0	0.1, **0.2**, 0.35, 0.5, 0.7, 1
Path half-width R	2.5 m	CMA-ES hidden units	4, 8, **16**, 24, 32
Altitude	1 m	NEAT population	8, 16, **24**, 32, 48
Network (CMA-ES)	12–16–3, tanh	NEAT compat. threshold	3.0
Genome dimension	259	NEAT node / conn. add prob.	0.2 / 0.5
Maximum tilt	34°	Generations (frontier / sens.)	200 / 150
Thrust authority	±40%	Seeds (frontier / sens.)	5 / 3

5. Results

Figure 2a shows the best population fitness at the nominal operating point, averaged over five seeds. Both methods start from the stable-hover prior. With near-zero network output, the drone remains near the centre while the reference moves towards the edges, giving a large initial tracking error. Both optimisers then improve rapidly. NEAT progresses slightly faster during the first ~ 30 generations, after which CMA-ES overtakes it and reaches a lower and more consistent final error. The mean champion cost is 0.19 ± 0.01 m for CMA-ES and 0.29 ± 0.03 m for NEAT. By 200 generations both methods have mostly converged, but CMA-ES shows less variation between seeds. Figure 2b compares the best flown trajectories with the reference. Both controllers follow the 5 m-wide figure-8 closely. Most visible error for both controllers occurs during the first lap, when the vehicle starts from rest while the reference is already moving.

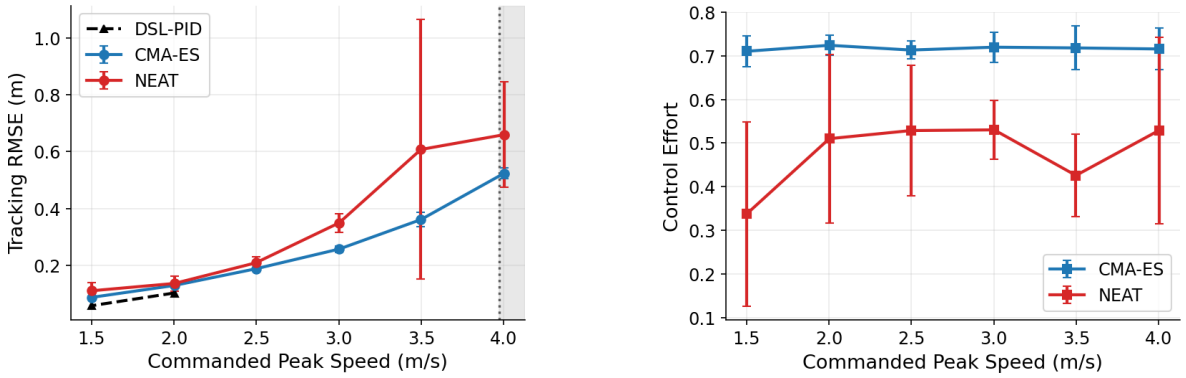


(a) Convergence

(b) Flown paths vs. reference

Figure 2: Learning and tracking at the nominal operating point ($T = 7.4\,\mathrm{s}$, 5 seeds) with error bars showing one standard deviation

Sweeping the lap time produces the accuracy results, which are displayed in Figure 3a and Table 2. As peak speed increases from $1.5\,\mathrm{m\,s^{-1}}$ to $4.0\,\mathrm{m\,s^{-1}}$, the Root-Mean-Square Error(RMSE) for tracking increases from roughly $0.09\,\mathrm{m}$ to $0.52\,\mathrm{m}$ for CMA-ES and from $0.11\,\mathrm{m}$ to $0.66\,\mathrm{m}$ for NEAT. Thus the agility–accuracy trade-off also appears under gradient-free evolution. None of the evolved controllers crashed during the clean frontier experiments. The inner loop maintains attitude stability, so higher speed mainly increases tracking error and control effort. In general, NEAT uses slightly less control effort than CMA-ES as shown in Figure 3b.



(a) Tracking accuracy

(b) Control effort (normalised |RPM − hover|)

Figure 3: Agility–accuracy frontier versus commanded peak speed. The shaded band marks the approximate actuation limit.

The physical feasibility of the trajectory provides an independent upper bound. For the figure-8, the peak reference acceleration is approximately $a_{\mathrm{peak}} \approx 2.1 R\omega^2 = 1.06\, v_{\mathrm{peak}}^2/R$. Maintaining altitude requires a bank angle $\phi_{\mathrm{req}} = \arctan(a_{\mathrm{peak}}/g)$ and thrust $f_{\mathrm{req}} = mg/\cos\phi_{\mathrm{req}}$. With the Crazyflie's $34°$ tilt limit, the approximate speed limit is:

$$v_{\mathrm{peak}} \lesssim \sqrt{g \tan 34° \, R/1.06}. \tag{9}$$

The limit grows with $\sqrt{R}$, so a wider track allows higher speeds. For the $5\,\mathrm{m}$-wide track used here, the sweep remains close to the feasible envelope, and the fastest network tracks the loop while slightly overshooting the tightest turns as displayed in Figure 4.

As a reference, the library's tuned DSL Proportional–Integral–Derivative (PID) controller [13] is more accurate at low speed, achieving about $0.06\,\mathrm{m}$ at $1.5\,\mathrm{m\,s^{-1}}$, and CMA-ES is within roughly $0.03\,\mathrm{m}$ of it. Above approximately $2.5\,\mathrm{m\,s^{-1}}$, however, the single PID controller loses control, while the evolved controllers remain stable up to $4.0\,\mathrm{m\,s^{-1}}$. The comparison is not fully matched, because each evolved controller is trained for one speed whereas the PID is a single general-purpose controller.

CMA-ES has the lower mean error at every tested speed and much smaller variation between seeds whereas NEAT is competitive but more variable. At $3.5\,\mathrm{m\,s^{-1}}$ one seed converges poorly and increases the standard deviation. The main difference is consistency rather than mean accuracy.

Table 2: Agility–accuracy frontier: tracking RMSE (mean ± s.d. over 5 seeds; best seed in parentheses). The tuned PID crashes for peak speed $\gtrsim 2.5\,\mathrm{m\,s^{-1}}$.

T (s)	Peak (m/s)	CMA-ES RMSE (m)	NEAT RMSE (m)	PID RMSE (m)
14.8	1.5	0.088 ± 0.01 (0.08)	0.111 ± 0.03 (0.06)	0.059
11.1	2.0	0.130 ± 0.01 (0.12)	0.137 ± 0.03 (0.11)	0.103
8.9	2.5	0.188 ± 0.00 (0.18)	0.209 ± 0.02 (0.18)	–
7.4	3.0	0.258 ± 0.01 (0.24)	0.350 ± 0.03 (0.32)	–
6.35	3.5	0.361 ± 0.03 (0.32)	0.608 ± 0.46 (0.35)	–
5.55	4.0	0.524 ± 0.02 (0.50)	0.660 ± 0.19 (0.49)	–

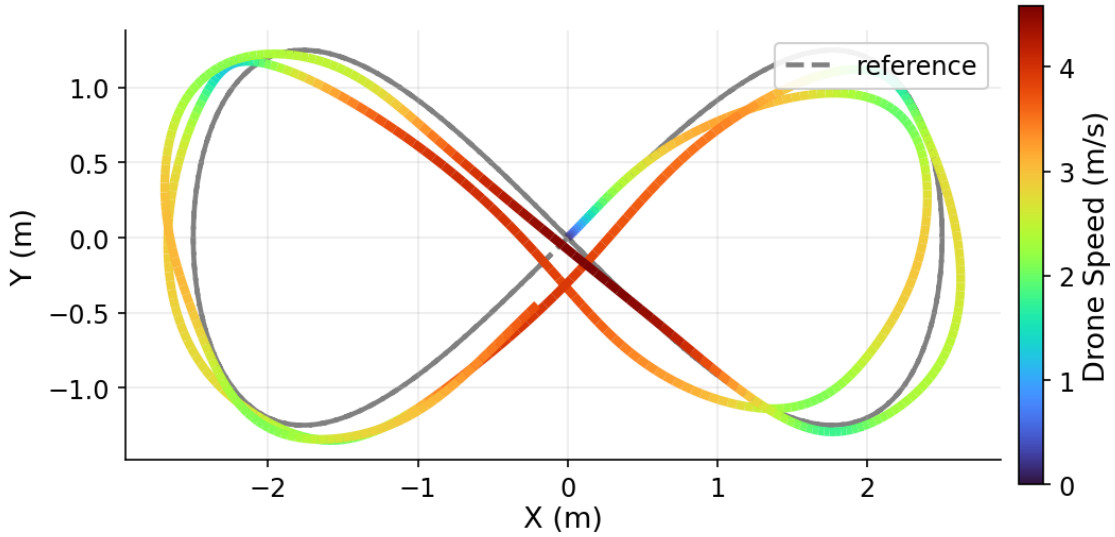
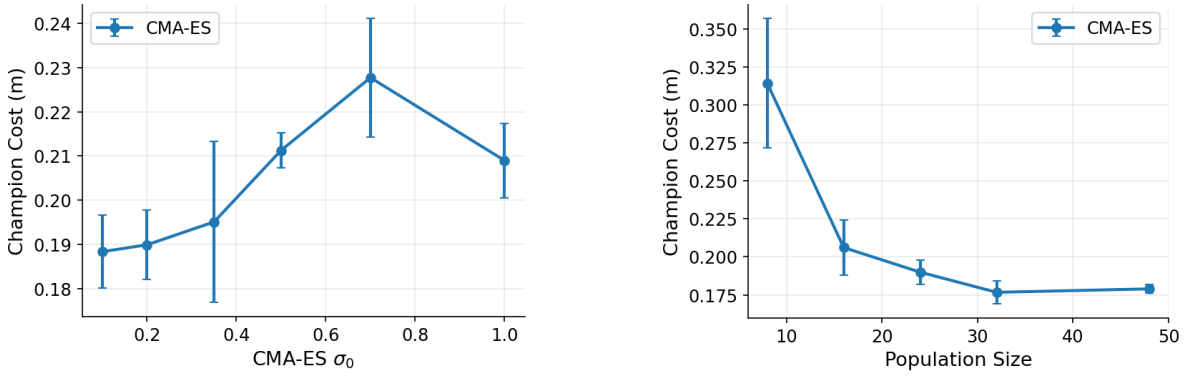


Figure 4: Flown path of the fastest champion ($4.0\,\mathrm{m\,s^{-1}}$), coloured by speed, against the reference.

Figure 5 varies the main hyperparameters around the nominal operating point. Population size has the largest effect. For CMA-ES, champion cost falls from $0.31\,\mathrm{m}$ at population 8 to about $0.18\,\mathrm{m}$ at population 32, with little further improvement at 48; NEAT improves from $0.75\,\mathrm{m}$ to $0.19\,\mathrm{m}$ over the same range. NEAT is therefore more dependent on population size, which is expected because it must search both weights and network structures. In practice, populations around 24–32 seemed to be sufficient, while populations below 16 perform poorly.

The CMA-ES initial step size has a smaller effect. The best result occurs at $\sigma_0 = 0.2$, while $\sigma_0 = 0.7$ gives the worst result. A large initial step can move candidates away from the region of stable controllers before covariance adaptation has identified useful search directions. Network width has little effect: CMA-ES performance stays between about $0.15\,\mathrm{m}$ and $0.19\,\mathrm{m}$ as the hidden layer changes from 4 to 32 units, and NEAT also tends to evolve small networks. This shows that the search is the main limitation instead of the controller capacity, specifically for this task.

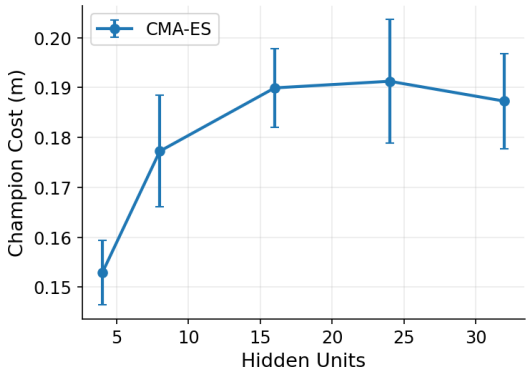


(a) CMA-ES step size

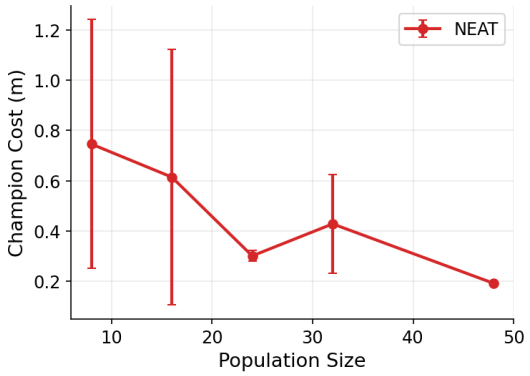
(b) CMA-ES population.

Figure 5: Sensitivity of champion cost to the main hyperparameters with mean ± s.d. over seeds

The neuroevolution occurs on clean, deterministic dynamics. To test generalisation and the reality gap [3, 10], the nominal-lap champions were evaluated without retraining under actuation and sensor disturbances. The main results of this are displayed in Figure 6. Actuation disturbance is the main weakness for both methods as a perturbation of 4% of hover thrust causes approximately 0–8% of flights to crash, 6% causes about 42% crashes, and at 8% almost all flights are lost. This shows that controllers evolved only on clean dynamics have little margin against unmodelled actuation errors. With



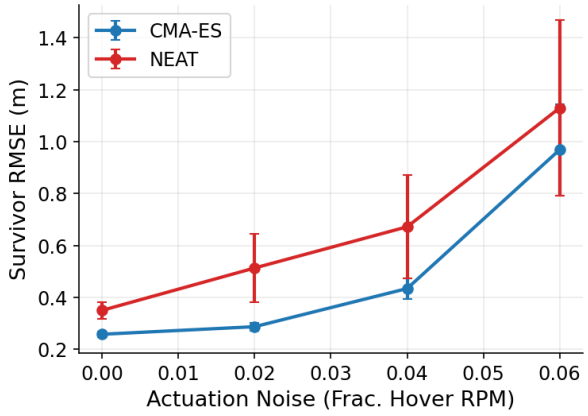
(c) CMA-ES network width



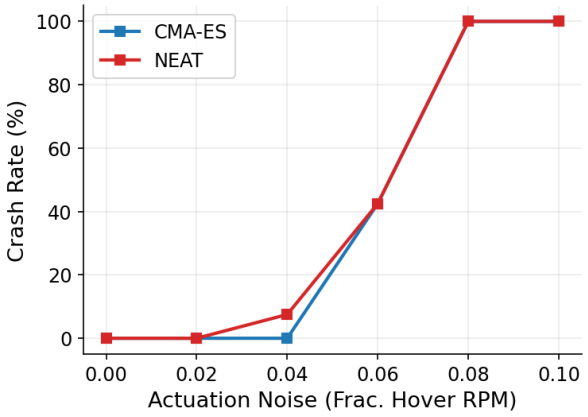
(d) NEAT population

Figure 5: Sensitivity analysis continued:

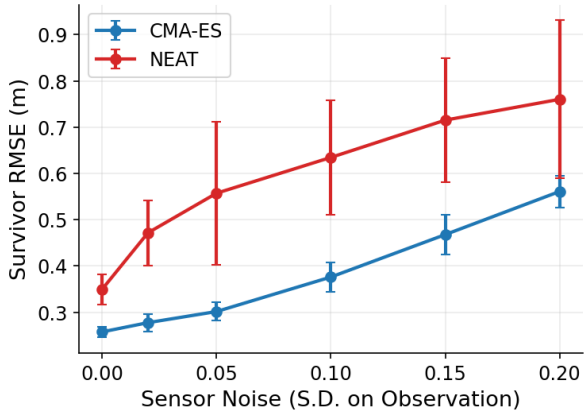
regards to sensor noise, CMA-ES has no crashes even at the largest tested perturbation ($\sigma = 0.20$), while NEAT reaches about 15% crashes and shows a larger increase in survivor error. On this task the fixed-topology CMA-ES controller is therefore more robust to sensor noise.



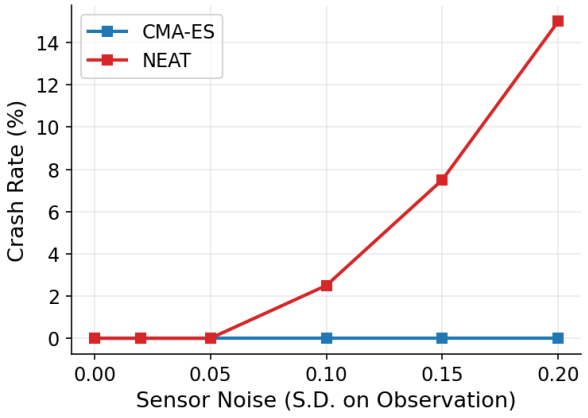
(a) Actuation disturbance accuracy



(b) Actuation disturbance crash rate



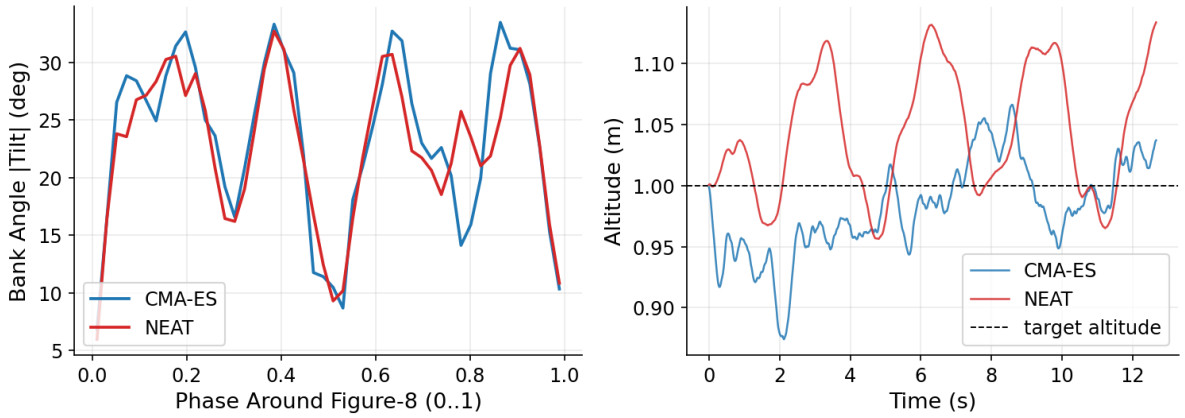
(c) Sensor noise accuracy



(d) Sensor noise crash rate

Figure 6: Robustness of trained champions to disturbances not seen during evolution, over 5 champions with eight noise realisations.

The controllers produce their largest tracking errors near the ends of the figure-8, where the turns are



(a) Bank angle around the figure-8

(b) Altitude over time

Figure 7: Evolved control strategy at $3.5\,\mathrm{m\,s^{-1}}$

tight and respond by increasing bank angle to approximately 30–33° as shown in Figure 7. At the agile $3.5\,\mathrm{m\,s^{-1}}$ lap both controllers also allow altitude to vary by up to about $0.1\,\mathrm{m}$ to maintain horizontal tracking, with CMA-ES tending to fall below the target altitude and NEAT to rise above it. NEAT also evolves a compact topology. Starting from 36 direct input-to-output connections, its champions at $3\,\mathrm{m\,s^{-1}}$ use about five hidden nodes and roughly 24 active connections on average, ranging across seeds from three to seven hidden nodes and 14–31 connections. NEAT therefore discovers a small architecture that varies between runs.

6. Conclusion

This report shows that both CMA-ES and NEAT can evolve stable quadrotor controllers for agile figure-8 tracking without gradients or a differentiable simulator. Both methods reproduce the agility–accuracy trade-off, where increasing speed increases tracking error and control effort even when the vehicle remains stable. This suggests that the trade-off is primarily a property of the control problem rather than of a specific optimisation method [3].

CMA-ES gives the stronger overall result in this report. It has lower mean error at every tested speed, substantially lower variation between seeds, and improved robustness to sensor noise, although the accuracy difference is not statistically significant with the available number of seeds. NEAT's main advantages are architectural flexibility and slightly lower control effort, but its solutions vary more between runs. Population size is the most important hyperparameter for both methods, with 24–32 individuals giving a good balance of performance and cost, and large network capacity is not needed for this task.

The robustness study shows a clear limitation of evolving only on clean simulation dynamics. Both methods are highly sensitive to actuation disturbances, while sensor noise has a smaller effect. This supports the need for domain randomisation, hardware adaptation, or hardware-in-the-loop training for more advanced systems [3, 2].

The report has several limitations. It is entirely simulation-based and uses one trajectory instead of more complex tracks, fixed inner-loop gains, and a fitness function focused on tracking error, and the injected disturbances test robustness but are not equivalent to hardware testing. Future work could introduce disturbances during evolution through domain randomisation [17], allow the trajectory speed and controller parameters to evolve together so that evolution selects the operating point on the agility–safety frontier, and add a reinforcement-learning baseline for a direct comparison between gradient-free evolution and gradient-based learning [6]. Additionally, the challenge of direct motor control could be reintroduced.

Bibliography

[1] Drew Hanover et al. 'Autonomous Drone Racing: A Survey'. In: *IEEE Transactions on Robotics* 40 (2024), pp. 3044–3067. DOI: 10.1109/TRO.2024.3400838.

[2] Elia Kaufmann et al. 'Champion-level drone racing using deep reinforcement learning'. In: *Nature* 620.7976 (2023), pp. 982–987. DOI: 10.1038/s41586-023-06419-4.

[3] Yunfan Ren et al. 'Learning Agile Quadrotor Flight in the Real World'. In: *Robotics: Science and Systems (RSS)*. arXiv:2602.10111. 2026.

[4] Daniel Mellinger and Vijay Kumar. 'Minimum snap trajectory generation and control for quadrotors'. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2011, pp. 2520–2525. DOI: 10.1109/ICRA.2011.5980409.

[5] Taeyoung Lee, Melvin Leok and N. Harris McClamroch. 'Geometric tracking control of a quadrotor UAV on SE(3)'. In: *49th IEEE Conference on Decision and Control (CDC)*. 2010, pp. 5420–5425. DOI: 10.1109/CDC.2010.5717652.

[6] Jemin Hwangbo et al. 'Control of a Quadrotor with Reinforcement Learning'. In: *IEEE Robotics and Automation Letters* 2.4 (2017), pp. 2096–2103. DOI: 10.1109/LRA.2017.2720851.

[7] Kenneth O. Stanley et al. 'Designing neural networks through neuroevolution'. In: *Nature Machine Intelligence* 1.1 (2019), pp. 24–35. DOI: 10.1038/s42256-018-0006-z.

[8] Tim Salimans et al. *Evolution Strategies as a Scalable Alternative to Reinforcement Learning*. arXiv:1703.03864. 2017.

[9] Stéphane Doncieux et al. 'Evolutionary Robotics: What, Why, and Where to'. In: *Frontiers in Robotics and AI* 2 (2015), p. 4. DOI: 10.3389/frobt.2015.00004.

[10] Nick Jakobi, Phil Husbands and Inman Harvey. 'Noise and the reality gap: The use of simulation in evolutionary robotics'. In: *Advances in Artificial Life (ECAL)*. 1995, pp. 704–720. DOI: 10.1007/3-540-59496-5_337.

[11] Rogier Koppejan and Shimon Whiteson. 'Neuroevolutionary reinforcement learning for generalized control of simulated helicopters'. In: *Evolutionary Intelligence* 4.4 (2011), pp. 219–241. DOI: 10.1007/s12065-011-0066-z.

[12] M. Mariani and S. Fiori. 'Design and Simulation of a Neuroevolutionary Controller for a Quadcopter Drone'. In: *Aerospace* 10.5 (2023), p. 418. DOI: 10.3390/aerospace10050418.

[13] Jacopo Panerati et al. 'Learning to Fly—a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control'. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2021, pp. 7512–7519. DOI: 10.1109/IROS51168.2021.9635857.

[14] Nikolaus Hansen and Andreas Ostermeier. 'Completely Derandomized Self-Adaptation in Evolution Strategies'. In: *Evolutionary Computation* 9.2 (2001), pp. 159–195. DOI: 10.1162/106365601750190398.

[15] Nikolaus Hansen. *The CMA Evolution Strategy: A Tutorial*. arXiv:1604.00772. 2016.

[16] Kenneth O. Stanley and Risto Miikkulainen. 'Evolving Neural Networks through Augmenting Topologies'. In: *Evolutionary Computation* 10.2 (2002), pp. 99–127. DOI: 10.1162/106365602320169811.

[17] Josh Tobin et al. 'Domain randomization for transferring deep neural networks from simulation to the real world'. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30. DOI: 10.1109/IROS.2017.8202133.